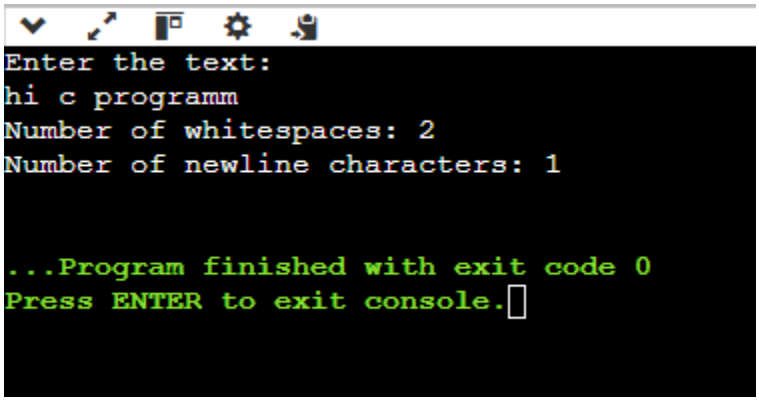
Experiment - 5

Design a lexical Analyzer to find the number of whitespaces and newline characters.

-> CODE :

```
#include <stdio.h>
int main()
{
   char str[200];
   int i, space = 0, newline = 0;
   printf("Enter the text:\n");
   fgets(str, sizeof(str), stdin);
   for(i = 0; str[i] != '\0'; i++)
   {
      if(str[i] == ' ' || str[i] == '\t')
         space++;
      else if(str[i] == '\n')
         newline++;
   }
   printf("Number of whitespaces: %d\n", space);
   printf("Number of newline characters: %d\n", newline);
   return 0;
}
```

OUTPUT :



Experiment - 6

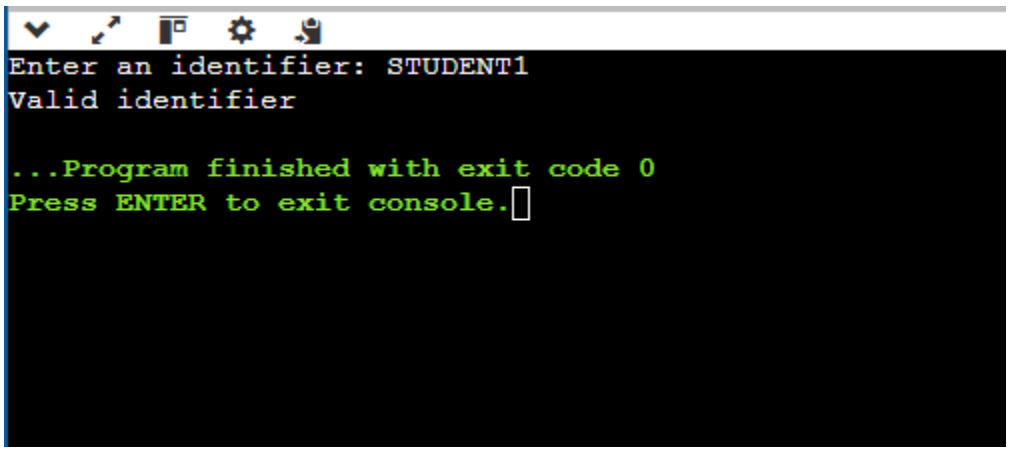
Develop a lexical Analyzer to test whether a given identifier is valid or not.

-> CODE :

```
#include <stdio.h>
#include <ctype.h>
int main()
{
   char id[50];
   int i, valid = 1;
   printf("Enter an identifier: ");
   scanf("%s", id);
   if (!isalpha(id[0]) && id[0] != '_')
      valid = 0;
   for (i = 1; id[i] != '\0'; i++)
```

```
    {
        if (!isalnum(id[i]) && id[i] != '_')
        {
            valid = 0;
            break;
        }
    }
    if (valid)
     printf("Valid identifier");
    else
        printf("Invalid identifier");
    return 0;
}
```

-> OUTPUT :



Experiment - 7

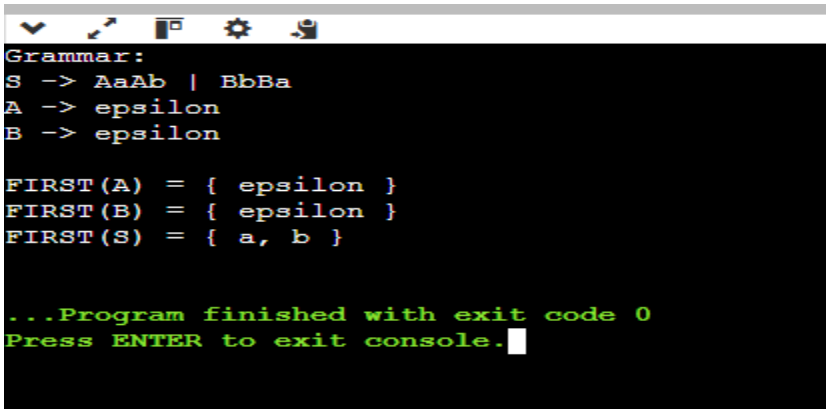
Write a C program to find FIRST( ) - predictive parser for the given grammar
S → AaAb / BbBa
A → ∈
B → ∈

-> CODE :

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
char prod[3][10] = {"AaAb", "BbBa", "e"};
void first(char c)
{
   int i;
   if (!isupper(c))
   {
      printf("%c ", c);
      return;
   }
   if (c == 'A' || c == 'B')
   {
      printf("e ");
      return;
   }
   if (c == 'S')
   {
```

```
        for (i = 0; i < 2; i++)
        {
            if (prod[i][0] == 'A')
                printf("a ");
            else if (prod[i][0] == 'B')
                printf("b ");
        }
    }
}
int main()
{
    printf("FIRST(A) = { ");
    first('A');
    printf("}\n");
    printf("FIRST(B) = { ");
    first('B');
    printf("}\n");
    printf("FIRST(S) = { ");
    first('S');
    printf("}\n");
    return 0;
}
```

OUTPUT :



Experiment - 8

Write a C program to find FOLLOW( ) - predictive parser for the given grammar
S → AaAb / BbBa
A → ∈
B → ∈

-> CODE :

```
#include <stdio.h>

char grammar[2][5] = {"AaAb", "BbBa"};

void follow(char ch)
{
   int i, j;

   if (ch == 'S')
      printf("$ ");

   for (i = 0; i < 2; i++)
   {
      for (j = 0; grammar[i][j] != '\0'; j++)
      {
         if (grammar[i][j] == ch)
         {
            if (grammar[i][j + 1] != '\0')
               printf("%c ", grammar[i][j + 1]);
            else
               follow('S');
         }
      }
   }
}
```

```
int main()
{
   printf("FOLLOW(S) = { ");
   follow('S');
   printf("}\n");

   printf("FOLLOW(A) = { ");
   follow('A');
   printf("}\n");

   printf("FOLLOW(B) = { ");
   follow('B');
   printf("}\n");

   return 0;
}
```

-> OUTPUT :

